عوالم البرمجة

أكاديمية عوالم البرمجة

Git & GitHub 101

من الصفر إلى أول Pull Request

.BEYOND CODE. BEYOND LIMITS

10

الوحدة 10

التراجع عن الأخطاء بأمان

استعد الملفات، وأصلح الـ commits، وتعافَ من الأخطاء دون ذعر.

التراجع عن التغييرات قبل الـ commit

Undo changes before you commit

أهداف التعلم

- أن تتخلص من تعديل ملف.
- أن تُخرج ملفًا من staging.
- أن تحفظ العمل جانبًا بـ stash.

المتطلبات المسبقة

الوحدة 4.

مقدمة

أغلب الأخطاء تحدث قبل الـ commit، وهي الأسهل إصلاحًا.

الشرح

`git restore index.html` يرمي تعديلاتك في ذلك الملف ويعيده إلى آخر commit. **لا يمكن التراجع عن ذلك** لأن Git لم يحفظ تلك التعديلات.

`git restore --staged style.css` يُخرج الملف من منطقة staging ويُبقي تعديلاتك. وهو عكس `git add`.

نفّذ `git status` أولًا لتعرف أي الملفات في staging وأيها لا.

`git stash push -m "work in progress"` يضع تعديلاتك غير المحفوظة في درج آمن ويعطيك شجرة نظيفة.
`git stash list` يعرض الأدراج و `git stash pop` يعيد الأحدث.

استخدم stash عندما تريد تبديل الفرع أو السحب ولست جاهزًا للـ commit.

في VS Code: مرّر المؤشر على ملف في Source Control؛ زر ↶ **Discard Changes** يستعيده، وزر **−** يُخرجه من staging. التجاهل يطلب تأكيدًا وهو أيضًا لا رجعة فيه.

تشبيه من الحياة

الـ restore ممحاة تعيد الصفحة إلى آخر نسخة مطبوعة. والـ stash أن تدفع مكتبك الفوضوي إلى درج لوقت لاحق.

مخطط توضيحي



الشكل 1 · رسم توضيحي: restore وإخراج من staging

الأوامر

`git restore FILE`

`git restore --staged FILE`

`git stash push -m "msg"`

`git stash pop`

الشكل 1 يعرض restore وإخراج الملف من staging.

أخطاء شائعة وحلولها

استخدام restore على عمل تريد الاحتفاظ به.	استخدم stash أو commit أولًا.
الخلط بين restore وrestore --staged.	الأول يرمي التعديلات والثاني يُخرج من staging فقط.
نسيان العمل المخزّن.	تحقق من `git stash list` .

تمرين عملي

عدّل index.html وتخلّص من التعديل. ثم أضف style.css إلى staging وأخرجه.

النتيجة المتوقعة

يعرض `git status` أن index.html نظيف وstyle.css معدّل وليس في staging.

اختبر فهمك

1. ماذا يفعل git restore FILE؟

- ○ يرفع
- ○ يرمي تعديلاتك
- ○ يحفظ commit

2. ماذا يفعل restore --staged؟

- ○ يُخرج من staging ويُبقي التعديلات
- ○ يحذف الملف
- ○ يرفع

3. git stash لـ...

- ○ النسخ
- ○ حفظ العمل جانبًا
- ○ حذف التاريخ

الإجابات

- يرمي تعديلاتك — لا رجعة فيه.
- يُخرج من staging ويُبقي التعديلات — يلغي git add.
- حفظ العمل جانبًا — pop يعيده.

الخلاصة

restore يرمي التعديلات، وrestore --staged يُخرج من staging، وstash يركن العمل.

مصطلحات أساسية

restore	إعادة ملف إلى حالة محفوظة
unstage	إخراج من منطقة staging
stash	تخزين مؤقت للتعديلات

الدرس التالي

الدرس 10.2 — إصلاح آخر commit.

تعديل commit أو عكسه

Fix or revert a commit

أهداف التعلم

- أن تعدّل آخر commit.
- أن تعكس commit.
- أن تختار بين amend وrevert.

المتطلبات المسبقة

الدرس 10.1.

مقدمة

أحيانًا يكون الخطأ داخل commit. والأداة الصحيحة تعتمد على أنك رفعته أم لا.

الشرح

لم يُرفع بعد. `git commit --amend -m "Better message"` يستبدل آخر commit. لإضافة ملف منسي نفّذ `git add file` أولًا ثم amend.

الـ amend **يعيد كتابة** الـ commit (يأخذ ID جديدًا). لا تعدّل commit قد يملكه غيرك.

رُفع أو شورك. `git revert HEAD` ينشئ **commit جديدًا** يلغي القديم. يبقى التاريخ صادقًا وآمنًا للجميع.

لعكس commit أقدم استخدم رقمه من `git log --oneline` : `git revert 8d2e5b1` .

قد يتعارض الـ revert؛ حلّه كما في الدرس 7.3 ونفّذ `git revert --continue` .

بعد الـ revert نفّذ `git push` كالمعتاد.

تشبيه من الحياة

الـ amend تصحيح خطأ إملائي في رسالة قبل بريدها. والـ revert إرسال رسالة ثانية تلغي الأولى.

مخطط توضيحي



الشكل 1 · رسم توضيحي: amend وrevert

الأوامر

`git commit --amend -m "msg"`

`git revert HEAD`

`git revert COMMIT-ID`

الشكل 1 يعرض الاثنين.

أخطاء شائعة وحلولها

amend بعد الرفع.	استخدم revert؛ الـ amend يحتاج force push وهو خطر.
عكس ID خاطئ.	تحقق من `git log --oneline` .
توقّع أن revert يحذف التاريخ.	هو يضيف commit جديدًا.

تمرين عملي

نفّذ commit برسالة فيها خطأ إملائي وعدّله بـ amend. ثم اعكس آخر commit.

النتيجة المتوقعة

يعرض `git log --oneline` الـ commit الـ revert في الأعلى.

اختبر فهمك

1. أيهما آمن للـ commits المرفوعة؟

- ○ git reset --hard
- ○ git revert
- ○ git commit --amend

2. ماذا يغيّر amend؟

- ○ آخر commit
- ○ كل الـ commits
- ○ الـ remote

3. هل يحذف revert التاريخ؟

- ○ محليًا فقط
- ○ لا
- ○ نعم

الإجابات

- git revert — يضيف commit جديدًا.
- آخر commit — يعيد كتابته.
- لا — يضيف commit يلغي.

الخلاصة

amend قبل الرفع، وrevert بعد الرفع.

مصطلحات أساسية

amend	استبدال آخر commit
revert	commit يلغي آخر
commit ID	رمز قصير مثل 8d2e5b1

الدرس التالي

الدرس 10.3 — Reset وreflog والأخطاء الشائعة.

Reset وreflog والأخطاء الشائعة

Reset, reflog and common errors

أهداف التعلم

- أن تشرح soft وmixed وhard.
- أن تتعافى بـ reflog.
- أن تصلح خمسة أخطاء شائعة.

المتطلبات المسبقة

الدرس 10.2.

مقدمة

الـ reset يرجّع فرعك إلى الخلف. هو قوي، فتعلّم أنماطه الثلاثة.

الشرح

`git reset --soft HEAD~1` يلغي آخر commit ويُبقي التغييرات **في staging**.

`git reset HEAD~1` (mixed وهو الافتراضي) يُبقي التغييرات في الملفات لكن **خارج staging**.

`git reset --hard HEAD~1` يلغي الـ commit **ويرمي التغييرات**. استخدمه فقط على عمل محلي لا تحتاجه.

لا تعمل reset لـ commits رفعتها؛ استخدم revert.

`git reflog` يسرد أماكن HEAD بما فيها commits تبدو ضائعة. استخدم `git reset --hard ID` أو `git switch -c rescue ID` للعودة (الشكل 1). الـ reflog محلي وتنتهي مدة إدخالاته بعد أسابيع.

أخطاء شائعة. «Author identity unknown» ← اضبط `user.name` و`user.email`. «not a git repository» ← `cd` إلى المشروع. «nothing to commit» ← لا تغيير أو لم يُضف. «failed to push some refs … fetch first» ← `git pull` ثم push. «detached HEAD» ← `git switch main` (أو `git switch -c name` للاحتفاظ بالعمل).

تشبيه من الحياة

soft وmixed وhard ثلاثة أعماق لإرجاع شريط: أبقِ كل شيء جاهزًا، أو أبقِ التعديلات، أو امسح.

مخطط توضيحي

رسم توضيحي · Git Bash (ليس لقطة حقيقية)

```
$ git reset --soft HEAD~1
$ git reset HEAD~1
$ git reflog
1fa09c3 HEAD@{0}: reset: moving to HEAD~1
8d2e5b1 HEAD@{1}: commit: Add about page
$ git reset --hard 8d2e5b1
HEAD is now at 8d2e5b1 Add about page
```

الشكل 1 · رسم توضيحي: reset وreflog

الأوامر

`git reset --soft HEAD~1`

`git reset HEAD~1`

`git reflog`

`git reset --hard ID`

الخيار --hard مدمّر للعمل غير المحفوظ.

أخطاء شائعة وحلولها

استخدام --hard بخفة.	فضّل soft أو mixed.
عمل reset لـ commits مرفوعة.	استخدم revert.
اليأس من commits ضائعة.	انظر في reflog أولًا.

تمرين عملي

نفّذ commit تجريبيًا وألغِه بـ reset --soft ثم أعد الحفظ.

النتيجة المتوقعة

الملف في staging من جديد بعد reset --soft.

اختبر فهمك

1. أي reset يُبقي التغييرات في staging؟

- ○ --mixed
- ○ --soft
- ○ --hard

2. كيف تجد commit «ضائعًا»؟

- ○ git reflog
- ○ git clone
- ○ git fetch

3. عند الرفع تعني «fetch first»...

- ○ أعد التثبيت
- ○ اسحب قبل الرفع
- ○ احذف المستودع

الإجابات

- --soft — الـ soft يُبقيها في staging.
- git reflog — يسجّل حركات HEAD.
- اسحب قبل الرفع — عند GitHub commits أحدث.

الخلاصة

الـ reset يرجّع محليًا، والـ reflog ينقذ، والـ revert للتاريخ المشترك.

مصطلحات أساسية

reset	نقل الفرع إلى commit آخر
reflog	سجل حركات HEAD
detached HEAD	HEAD على commit لا على فرع

الدرس التالي

الوحدة 11 — عادات احترافية.

المختبر العملي 10 · تراجع بأمان

1. عدّل `index.html` ونفّذ `git restore index.html` . تأكد أن التعديل اختفى.
2. عدّل `style.css` ونفّذ `git add style.css` ثم `git restore --staged style.css` . تحقق بـ `git status` . ثم استعده.
3. أنشئ `test.txt` واحفظه `Add test file` ثم نفّذ `git commit --amend -m "Add test file for practice"` .
4. نفّذ `git revert HEAD --no-edit` .
5. أنشئ commit آخر ثم `git reset --soft HEAD~1` وأعد حفظه.
6. نفّذ `git reflog` وابحث عن commits.

الفحص	ينجح إذا...
Restore	رُميت التعديلات كما هو متوقع
Amend/revert	يعرض السجل commit الـ revert
Reset/reflog	أعدت الحفظ ووجدته في reflog

معيار التقييم: 3 من 3 = نجاح.

اختبار الوحدة 10 (علامة النجاح: 70%)

1. أي أمر يرمي تعديلات ملف؟

- ○ git add FILE
- ○ git push
- ○ git restore FILE

2. أيهما آمن للـ commits المرفوعة؟

- ○ git commit --amend
- ○ git revert
- ○ git reset --hard

3. reset --soft يُبقي التغييرات...

- ○ في staging
- ○ محذوفة
- ○ في الـ remote

4. كيف تجد commit ضائعًا؟

- ○ git config
- ○ git reflog
- ○ git init

5. الرفع يقول «fetch first». ماذا تفعل؟

- ○ تحذف المستودع
- ○ force push
- ○ git pull ثم push

الإجابات

- git restore FILE — لا رجعة فيه.
- git revert — يضيف commit جديدًا.
- في staging — تبقى في staging.
- git reflog — يتتبع HEAD.
- git pull ثم push — احصل على الأحدث أولًا.